

Graded Project: Orchestration and Scaling

Welcome to your graded project on **container orchestration and scaling**. In this project you'll take a real multi-service MERN application, containerize it, ship it through a Jenkins CI pipeline to Amazon ECR, and deploy it to a production-style Kubernetes cluster on AWS (EKS) with monitoring, logging, and (optionally) ChatOps alerts layered on top.

Treat this as an end-to-end build: by the time you submit, the app should actually be running on your cluster, not just described in a doc.

1. What You're Working With

Jenkins (provided for you)

URL	https://jenkinsacademics.herovired.com/
Username	herovired
Password	herovired@123

This is a shared academic Jenkins instance use it to build and run your CI pipeline.

Project Repository

- Repo: github.com/UnpredictablePrashant/StreamingApp

This is the MERN-based application you'll be deploying throughout the project.

2. Project Workflow

Work through the steps below in order each one builds on the last.

Step 1 Version Control with Git

- Fork the main repository into your own GitHub account.
- Keep your fork in sync with the main repository as it updates, so you're always working from a current base.

Step 2 Prepare the Application

a) Containerize the application

- Write a Dockerfile for the frontend.
- Write a Dockerfile for each backend service.

b) Push images to Amazon ECR

- Build a Docker image for every component (frontend + each backend service).
- Create a dedicated ECR repository per component.
- Push each image to its matching ECR repository.

Step 3 AWS Environment Setup

- Install and configure the AWS CLI with your own AWS access credentials.

Step 4 Continuous Integration (CI) with Jenkins

a) Set up Jenkins

- Install Jenkins on an AWS EC2 instance.
- Install and configure the plugins and credentials your pipeline needs (Git, Docker, AWS/ECR credentials, etc.).

b) Build your Jenkins pipelines

- Create pipeline jobs that build each Docker image and push it to ECR.
- Configure the pipeline to trigger automatically on new commits to your Git repository.

Step 5 Kubernetes Deployment on EKS

a) Create an EKS cluster

- Provision an Amazon EKS cluster using `eksctl` (or an equivalent tool).

b) Deploy with Helm

- Package the MERN stack as a Helm chart.
- Deploy it to your EKS cluster via `helm install` / `helm upgrade`.

Step 6 Monitoring and Logging

a) Monitoring

- Set up Amazon CloudWatch to collect metrics and configure alarms for your services.

b) Logging

- Centralize application logs using CloudWatch Logs (or a comparable centralized logging solution).

Step 7 Documentation

Document your entire system, including:

- Overall system architecture (with diagrams).
- Step-by-step deployment process.
- Configuration details and any scripts you wrote.

Upload this documentation to your GitHub repository.

Step 8 Final Validation

- Confirm that both the frontend and backend are fully functional and reachable.
- Confirm the deployment meets the project's scaling requirements (e.g., the app stays healthy under scaling operations).

3. Bonus Step 9: ChatOps Integration

This step is optional and for extra credit.

- **Create SNS topics** for key pipeline/deployment events (e.g., deployment success, deployment failure).
- **Integrate with a messaging platform** connect your SNS topics to Slack, Microsoft Teams, or Telegram so your team gets real-time alerts.

4. Submission Instructions

1. Make sure your project is complete and fully tested end-to-end.
2. Push all code, Helm charts, pipeline configuration, and documentation to your GitHub repository.
3. Create a short text, Word, or PDF file containing the link to your repository.
4. Upload that file via Vlearn.